

ASSEMBLY



By: Maryam Eid :)

L04: Requirements for Coding in Assembly Language

ما زلنا مكملين على نفس برنامج المحاضرة اللي فاتت اللي هو ال Debug فما تنساش تنزله عندك وتجرب ال commands بنفسك

PTR Operator

حجم ال data movement بيتحدد بحجم ال register، يعني مثلاً لو هنقل داتا من ميموري ل register أيًا يكن مين ال source ومين ال destination ففي كلا الحالتين حجم الداتا اللي هتتنقل هتبقى حسب حجم ال register يعني لو AL مثلاً هيتنقل 1 byte ولو AX يبقى 2 bytes

بس فيه حالات ال CPU بيبقى confused ومش عارف ينقل قد إيه من الداتا زي مثلاً لو مافيش register وعازب ينقل immediate value لميموري فوقتها أنا هحتاج بشكل explicit أحدد له حجم ال data movement وده عن طريق ال PTR operator

```
MOV AX, [120]
```

هنا الكود تمام لأن ال CPU بالفعل عارف حجم ال register فهو هيروح ياخد 2 bytes بدايةً من العنوان ده

خلينا نشوف مثال هحتاج فيه ال PTR

```
MOV [120], 25
```

الكود هنا غلط لأن أنا بقول لل CPU انقل القيمة 25 بس ما عرفتوش ولا هو عارف حجمها كام هل ينقلها 1 byte ولا 2 bytes

```
MOV WORD PTR [120], 25
```

الكود ده تمام لأن هنا حتى لو ال 25 كقيمة بالهيكسا حجمها 1 byte بس أنا كتبت WORD PTR فأنا بقول صراحةً لل CPU انقلها ك 2 bytes فوقتها فرق المساحة اللي هو هنا حجمه 1 byte هيتملأ بأصفار على الشمال هتتنقل 0025 بس ما ننساش إننا شغالين little indian هتتخزن في الميموري معكوسة كده

```
120: 25
121: 00
```

مثال تاني ممكن نحتاج فيه ال PTR

```
1. .DATA
2. Var DWORD 32E46FFA
3. .CODE
4. MOV AX, var
```

هنا error أكيد لأن ما ينفعش أنقل متغير حجمه 4 bytes ل register حجمه 2 bytes

```
1. .DATA
2. Var DWORD 32E46FFA
3. .CODE
4. MOV AX, WORD PTR var
```

بكل بساطة لما استعملت WORD PTR فأنا قلت لل CPU مش عايزك تنقل ال 4 bytes كلهم انقل 2 bytes بس وده نفس حجم ال register فمش هيحصل مشكلة وعشان little indian هينقل 6FFA بس

Assembly Syntax

والوقت هنبدا في برنامج تاني اسمه ال TASM فبرضو ما تنساش تنزله وتجرب بنفسك ما تذاكرش المحاضرة بشكل نظري بس

Comments

عشان أكتب comment ببدأ ب “;” semicolon ووقتها ال assembler هيتجاهله ومش هيد generate له ماشين كود

```
MOV AX, 5 ; load the value 5 into the AX register
```

Reserved Words

Statement Type	Example	Explanation
Instructions	MOV, ADD	ببساطة هي العمليات اللي يقدر ال CPU ينفذها
Directives	END, SEGMENT	عبارة عن reserved word بتوجه ال assembler زي تعريف داتا أو حجز أماكن في الميموري لكن هي مش أوامر بينفذها ال CPU ولا بتتحول لماشين كود
Operators	FAR, SIZE	مجرد بتساعد ال assembler في تنفيذ ال instructions ومش بتتحول لماشين كود
Predefined Symbols	@data, @model	دي حاجات بتبقى متعرفة جوا ال assembler نفسه تسهيلات للكود

Identifier

هو مجرد اسم بيد reference لمكان في الميموري وله نوعين:

- Name وده اسم متغير

```
COUNTER DB 0
```

منا ميروح يحجز 1 byte في الميموري ويربط اسم المتغير اللي هو COUNTER بعنوان المكان ده

- Label وده اسم ل instruction أو procedure أو segment

```
1. start:
2. MOV AX, @data
3. MOV DS, AX
```

شروط التسمية:

- يمكن أستعمل أي حروف (A-Z, a-z) وأي أرقام (0-9) بس ما أبدأش برقم
- يمكن أستعمل special characters زي ؟ و \$ و @ بس دي يفضل ما نبدأش بها عشان فيه predefined symbols في ال assembler بتبدأ بها
- و. بس ما ينفعش أبدأ بها

Instructions

[identifier] **Operation** [operand(s)] [; comment]

- أي instruction لازم تحتوي على **operation** لكن باقي الأجزاء optional
- عدد ال operands ممكن يكون صفر أو أكثر
 - 0 operand
 - RET
 - 1 operand
 - INC **CX**
 - 2 operands
 - ADD **CX**, 25

Directives

Directive	Description	Explanation
PAGE [length] [,width]	Length → # lines per page Width → # chars per line Default → 50, 80	مجرد بعّرف ال assembler كمساعدة الكود مكتوب في كام سطر وكل سطر فيه كام حرف
TITLE text		أقدر أعتبره زي comment كده باسم البرنامج ومالوش أي تأثير على التنفيذ مجرد حاجة توضيحية للمبرمج مش ال assembler
segment-name SEGMENT [align] [combine] ['class']	segment-name → identifier SEGMENT → reserved word (start) Align → segment size, default → PARA (64KB) Combine → how to combine with other segments 'Class' → group related segments when linking	هنا بعّرف segment سواء data, code, or stack ويحدد تفاصيله في ال combine ممكن أستعمل PUBLIC أو COMMON أو NONE وغيرهم
segment-name ENDS	segment-name → same identifier ENDS → reserved word (end)	Public بيجمع كل ال segments اللي لها نفس الاسم كأنها segment واحد
Procedure-name PROC FAR / NEAR	Procedure-name → identifier PROC → reserved word (start) FAR → program entry point	أي برنامج لازم يبقى له FAR procedure واحد بس عشان البرنامج يبدأ تنفيذ من عنده وأي عدد من NEAR procedures ال

Procedure-name ENDP	Procedure-name → same identifier ENDP → reserved word (end)	
END [procedure-name]	procedure-name → Name of the procedure designated as FAR	هنا بنهي البرنامج ولو كانت END بس بدون حاجة معنى كده إن البرنامج ده مش executable وما فيش فيه instructions وممكن يكون فيه داتا بس مثلاً
ASSUME SS: stackname, DS: datasegname, CS: codesegname		في البرنامج عندي بس 3 segments ويقدر أعرف ال segments بنفسه فيستعمل assume عشان أربط ال segments اللي أنا عرّفتها بال data, code, or stack segments
.286 .386 .486	.286 → POPA - PUSH .386 → MOVX/MOZX - SHLD/SHRD .486 → CMPXCHG - XADD	كل CPU له instructions set خاصة به ولو عايز أستعمل instructions خاصة بـ CPU أحدث فبكتب اسمه قبل ال code segment

Simplified Segment Directives

تسهيلًا للكود ممكن أستعمل simplified segment directives ودي لما ال assembler يشوفها فهو هيفهم نوع ال segment وهي generate بدايته ونهايته بشكل تلقائي وكذلك ال assume statements بس قبل ما أستعملها لازم أستعمل directive .MODEL

.MODEL memory-model

Model	# Code Segments	# Data Segments
Small	$I \leq 64K$	$I \leq 64K$
Medium	Any number, any size	$I \leq 64K$
Compact	$I \leq 64K$	Any number, any size
Large	Any number, any size	Any number, any size
Huge	Any number, any size	Any number, any size
Tiny	يستعمله في برامج ال .COM. ويبقى فيه segment واحد بس لل code, data, and stack	

ومش هتحتاج تكتب SEGMENT وENDS هتكتب على طول:

- override له تعمل له 1024 bytes default size وال .STACK [size]
- .DATA
- .CODE [segment-name]

ومش هتحتاج تكتب اسم ال segment وال default segment names هي:

- STACK
- _DATA
- _TEXT

عشان أعمل initialization لل data segment في ال DS أقدر أستعمل @model ودي reserved word بتشاور على address ال data segment

```
1. MOV AX, @data
2. MOV DS, AX
```

وممكن أستغنى عن السطرين دول وأستعمل .STARTUP

وعشان أنهى البرنامج فأنا بكتب

```
1. MOV AX, 4C00H
2. INT 21H
```

وممكن أستغنى عنهم وأستعمل .EXIT

Defining Data Types

[name] Dn expression

- Name اسم المتغير
- Dn عبارة عن ال data type
 - DB (Define Byte) → 1 byte
 - DW (Define Word) → 2 bytes
 - DD (Define Doubleword) → 4 bytes
 - DQ (Define Quadword) → 8 bytes
 - DT (Define Ten Bytes) → 10 bytes
- Expression عبارة عن ال initial value ويمكن يبقى initialized وأكتب قيمة أو unutilized وأكتب ؟

Numeric Constants

- 1 B → binary
- 12 D → decimal (default)
- 12 H → hexadecimal (first digit must be 0 - 9)
- 12.4 R → real value

EQUATE Directives

```
Val = 5
Val EQU 5
```

سطرين الكود equivalent لبعض